Proposal for the Structural Hardening of Stela Governance Architecture and the Implementation of a Pointer-First Agent Constitution

1. Executive Summary: The Divergence of Intent and Execution

The Stela repository operates under a governance model defined by the *Policy of Truth* (PoT.md), which establishes a strict hierarchy of authority: the Operator (Human) serves as the sovereign, the Integrator (Lead AI) as the architect of governance, and the Contractor (Guest AI) as the executor of logic.[1] Theoretically, this system is designed to be "Pointer-First," meaning that agents should possess minimal intrinsic knowledge and rely exclusively on pointers to immutable sources of truth (SSOT) to navigate and execute tasks. However, a comprehensive forensic audit of the current repository state—specifically the active work surface TASK.md and the prompt engineering artifacts in docs/ops/prompts/—reveals a critical divergence between this constitutional intent and the operational reality.

The core failure vector identified is the structural weakness of TASK.md. While PoT.md establishes "Physical Laws" to prevent "Drift," TASK.md is explicitly exempted from the "Dirty State" rule (Section 2, PoT.md), allowing it to exist in a persistent state of flux.[1] This exemption, while operationally necessary to permit real-time logging, has inadvertently turned TASK.md into an unregulated accumulation point for "Semantic Debris"—residual context, obsolete objectives, and ghost data from previous sessions.[1] Because TASK.md fails to enforce a rigid, self-cleansing schema, the system has been forced to rely on "Prompt Engineering" hacks—brittle, repetitive instructions embedded in E-PROMPT-XX files—to artificially simulate the governance that the architecture fails to provide natively.[1]

This report proposes a radical architectural refactoring to align the repository with the user's demand for a "Pointer-First agent constitution" with "no duplicated canon text." The proposal outlines the transformation of TASK.md from a passive scratchpad into a "Hybrid-Hardened" Active Contract. This new contract will autonomously enforce "Freshness Gates," "Negative Constraints," and "Thread Transitions," thereby rendering the prompt hacks obsolete. Furthermore, the report defines the missing "Formally Robust Results" standard, moving verification from subjective narrative summaries to machine-verifiable, quantitative metrics.[1] The following analysis is exhaustive, covering the forensic diagnosis of current failures, the cognitive psychology of LLM adherence, and the specific technical specifications for the

proposed remedial architecture.

2. Forensic Audit of the Current Operational State

To understand "WHAT IS GOING ON WITH TASK.md" (User Query 1), one must look beyond the file itself and analyze the ecosystem of constraints, failures, and dependencies that surround it. The audit leverages the repository dump (dump-full-main-4b581538.txt) to reconstruct the operational reality.

2.1. The Dirty State Paradox and the Drift Vector

The *Policy of Truth* serves as the constitution of the Stela repository. Section 2 ("Enforcement") explicitly defines "System Failure States" that require an immediate "STOP WORK" response. Among these is the "Dirty State," defined as "Uncommitted changes in main".[1] This rule is the primary defense against entropy; it ensures that the canonical state of the repository is always known, hashed, and immutable.

However, the policy contains a specific, critical carve-out: "**except TASK.md**".[1] This exemption creates the "Dirty State Paradox."

- **The Necessity:** Agents require a mutable surface to "think," plan, and record progress. If TASK.md were subject to strict dirty-state locking, an agent could not update its checklist without triggering a system failure.
- **The Vulnerability:** By exempting TASK.md from the rigid hygiene enforced on the rest of the repo, the system effectively creates a "Drift Vector." TASK.md becomes the path of least resistance for entropy. Unlike code files which are scrubbed during Pull Request (PR) reviews, TASK.md persists across sessions, accumulating "Semantic Debris".[1]

The audit reveals that current operations fail to vigorously scrub this file between dispatch packets. A "Freshness Gate" exists in theory (Section 3.1 of the current TASK.md), but it only validates the *git branch* and *commit hash*.[1] It does not validate the *cleanliness of the instruction set*. Consequently, an agent initializing a new session often inherits the "Ghost Context" of the previous session—objectives, constraints, or "Work Logs" that are no longer relevant but exist in the context window because they physically reside in the "Active Work Surface".[1]

2.2. The Compliance of "Weak Ass" TASK.md

The user describes TASK.md as "weak ass." Forensically, this weakness translates to a lack of **Schema Rigidity**.[1]

- **Narrative vs. Imperative:** The current TASK.md relies on narrative descriptions of work (e.g., "Draft the implementation details") rather than imperative, verifiable data structures. Narrative text is open to interpretation by Large Language Models (LLMs), leading to "Hallucinated Compliance" where the agent believes it has satisfied a vague

requirement.

- **Placeholder Permissiveness:** The current templates likely use loose placeholders (e.g., <TBD>, [Insert Goal Here]). In the absence of a strict validator, LLMs (particularly ChatGPT) have a high propensity to "fill in the blanks" with plausible but hallucinated data to please the user, rather than stopping to report a missing input.[1]
- **Missing Negative Constraints:** While the *Policy of Truth* defines what *must* happen, the current TASK.md lacks a dedicated section for "Negative Constraints"—what *must not* happen.[1] This forces the Operator to inject these constraints via prompts ("Don't do X") rather than having them exist as immutable laws within the contract itself.

2.3. The "Gate Surface" Dependency Chain

The repository attempts to police TASK.md through external automation rather than internal structure. The workflow .github/workflows/task_policing.yml defines a list of "Gate Surfaces":

- ops/bin/open
- ops/bin/dump
- tools/lint/dp.sh
- .github/copilot-instructions.md
- docs/MANUAL.md

If any of these files are modified, the workflow fails unless TASK.md is also updated.[1] This proves that TASK.md is architecturally coupled to the governance logic. However, this coupling is reactive, not proactive. It ensures TASK.md is *touched*, but it does not verify that TASK.md is *correct*. The content of tools/lint/dp.sh (which presumably validates TASK.md) is critical here. While the code is missing from the dump, the user's complaint ("whole sections from TASK.md are missing") suggests that dp.sh is currently enforcing a loose or obsolete schema, failing to catch the structural degradation of the active contract.[1]

3. The Pathology of Prompt "Hacks" and Redundancy

The user asks: "why do my prompts... still have to include 'hacks' to make sure there is fencing or allowlist bs?" (User Query 3). The existence of these hacks is a direct symptom of the failure of TASK.md to serve as a self-enforcing constitution.

3.1. Deconstructing the "Hacks"

The prompts in docs/ops/prompts/ (E-PROMPT-01 through 04) are not merely instructions; they are defensive countermeasures against a fragile system.

Hack 1: The "Logic Fence" (Context Control)

- **The Hack:** Prompts like E-PROMPT-02 likely contain instructions wrapped in delimiters (e.g., triple backticks) with explicit commands: "You have no knowledge of files outside these fences".[1]

- **The Pathology:** This is a psychological trick played on the LLM. It attempts to override the model's training data or its tendency to "invent" files by artificially constraining its world view.
- **Why it exists:** It exists because TASK.md does not explicitly define the **Scope Boundary**. If the contract (TASK.md) clearly listed the Target Files allowed for modification, the agent would not need to be psychologically "fenced" in the prompt; it would be contractually bound by the file list.[1]
- **Why it fails:** As context windows grow (e.g., Gemini's 1M+ tokens), "Attention Dilution" renders these initial prompt fences less effective. The model "forgets" the fence as it processes the massive dump. A "Hardened" TASK.md places the fence *at the point of execution* (inside the task list), ensuring it is locally relevant when the agent is acting.[1]

Hack 2: The "Allowlist Boilerplate" (Jurisdictional Enforcement)

- **The Hack:** Prompts essentially copy-paste the "Filing Doctrine" from PoT.md, listing ops/ as Run, docs/ as Explain, etc., and manually instructing the agent not to cross these lines.[1]
- **The Pathology:** This violates the "No Duplicated Canon Text" rule. It creates a "Split Brain" problem where the rules exist in PoT.md and in E-PROMPT-02. If PoT.md changes (e.g., a new directory is added), the prompt becomes stale ("Canon Drift").
- **The Solution:** The prompt should simply say: "Enforce the Filing Doctrine defined in PoT.md Section 1.1." The *knowledge* of the doctrine should come from the pointer, not the prompt text.

Hack 3: The "Linguistic Precision" Repetition

- **The Hack:** Both .github/copilot-instructions.md and PoT.md explicitly list the ban on contractions and the requirement for quantitative reporting.[1]
- **The Pathology:** This redundancy acknowledges that the system does not trust the agent to read PoT.md effectively. It forces the behavioral standard into the system prompt to guarantee adherence.
- **The Fix:** By placing the "Behavioral Logic Standard" pointer in the **## I. REQUIRED CONTEXT LOAD** section of TASK.md, we force the agent to ingest the rules as part of its active memory for the specific task, removing the need to hard-code them in the prompt.

3.2. Context Window Saturation and Attention Dilution

The reliance on these hacks contributes to "Context Window Saturation." By stuffing the system prompt with defensive text, we reduce the available inference capacity for the actual task. Furthermore, "Attention Dilution" occurs when critical constraints are buried in a wall of boilerplate text.[1]

The "Pointer-First" architecture solves this by maximizing the signal-to-noise ratio. The prompt becomes minimal ("Execute TASK.md"), and TASK.md contains only the high-signal

pointers relevant to the immediate objective.

4. The "Formally Robust Results" Gap Analysis

The user asks: "where is the formally robust results file?" (User Query 2). The forensic analysis indicates that this file **does not exist as a static template** in the current repository state. Instead, it is an emergent artifact that *should* be generated by the execution of a hardened Dispatch Packet (DP), but currently lacks a rigorous definition.[1]

4.1. Definition of "Formally Robust"

Based on the governance analysis, a result is only "Formally Robust" if it satisfies the criteria of **"Zero Vibes"** and **"Machine Verifiability"**.[1]

- **Zero Vibes:** The output must contain zero narrative fluff, subjective adjectives (e.g., "successfully," "robustly," "cleaned up"), or conversational filler.
- **Machine Verifiability:** The output must consist of raw data that can be piped into a verification script (e.g., git status --porcelain).

4.2. Current State vs. Robust State

- **Current State (DP-OPS-0044-RESULTS.md):** This file represents a prototype of robustness. It includes "Verification Outputs (Raw)" and "Git Status".[1] However, it is likely generated ad-hoc by the agent based on loose instructions.
- **Robust State (The Requirement):** The "Formally Robust Results" must be a **Mandatory Section (Section 2.3)** within the TASK.md schema itself.[1] It forces the agent to populate specific fields:
    - **Logic Integrity Score:** A calculated metric (0-100%) of adherence.
    - **Drift Detected:** A binary Boolean (YES/NO).
    - **Protocol Violations:** An integer count (0).
    - **State Checksum:** The SHA-256 hash of the final state.

The "Formally Robust Results" file is not a template you *find*; it is a template you must *enforce* the creation of via the TASK.md contract.

5. Cognitive Architecture Analysis: Optimization for LLM Adherence

The redesign of TASK.md must account for the divergent cognitive architectures of the primary agents: **ChatGPT** (Document-Centric) and **Gemini** (Data-Centric). The current "weak" TASK.md fails to optimize for either, leading to friction.[1]

Table 1: Cognitive Model Requirements and TASK.md Optimization

Feature	ChatGPT Requirement (Document-Centric)	Gemini Requirement (Data-Centric)	Hybrid-Hardened TASK.md Solution
Hierarchy	Relies on visual headers (#, ##, ###) to determine scope and context switching.	Ignores visual nesting; parses data as flat key-value pairs.	**Strict Decimal Schema** (e.g., 3.1, 3.1.1). Provides sequence for ChatGPT and unique keys for Gemini.
Instructions	Prone to "Instruction Skipping" in long lists. Needs checklist format.	Prone to "Attention Dilution" in large contexts. Needs explicit delimiters.	**Unified Format:** - **Key:** Value. Combines bullet (ChatGPT) with Key (Gemini).
Placeholders	Hallucinates plausible text into <TBD> or loose brackets.	Treats empty fields as "null" or ignores them, leading to missing data.	**Sanitized Tokens:** ENTER_VALUE. Acts as a "Stop Condition" for ChatGPT and a schema violation for Gemini.
Context	Needs narrative flow to understand "why" before "what."	Needs "Context Fencing" to prevent external lookup (e.g., Google Search).	**Context Load Section:** Uses pointers to explain "why" (docs) and negative constraints to fence "what."

5.1. The Failure of the "Placeholder Paradox"

The current TASK.md likely contains boilerplate placeholders like DP-OPS-XXXX. This creates a logical deadlock. The PoT.md forbids "Ambiguity" and "Invention," yet the template *requires* the agent to invent a DP ID or fill in a placeholder. This conflict causes "Analysis Paralysis" or

non-deterministic behavior.[1]

- **The Fix:** The hardened TASK.md must be pre-filled by the **Integrator** (Lead AI) or the **Operator** (Human) before the **Contractor** touches it. Or, it must use specific "Sanitized Tokens" that are whitelisted in the linting rules.

6. Proposed Architecture: The Pointer-First Constitution

To "MAKE IT RIGHT" (User Query 5), we must implement a **Pointer-First Constitution** that resides entirely within TASK.md. This document will act as the single source of authority for the active session, rendering prompt hacks redundant.

6.1. Architectural Axioms

1. **No Duplicated Canon:** TASK.md shall contain zero rules. It shall contain only pointers to PoT.md and SoP.md.
2. **Self-Enforcing Schema:** The structure of TASK.md shall be decimal-indexed and machine-parseable.
3. **Clean-After-Use:** The document must contain a "Thread Transition" mechanism to scrub semantic debris.

6.2. The New TASK.md Structure (Detailed Analysis)

Section 0: FRESHNESS GATE (The Anchor)

- **Why:** To prevent the "Dirty State Paradox" where agents work on stale branches.
- **Mechanism:** Requires explicit validation of Base HEAD against the live git repo. If they do not match, the agent must STOP. This replaces the "Freshness" check in E-PROMPT-04.

Section I: REQUIRED CONTEXT LOAD (The Pointers)

- **Why:** To implement "Pointer-First" logic.
- **Mechanism:** A mandatory list of files the agent *must* read.
  - PoT.md (The Law)
  - SoP.md (The History)
  - docs/MANUAL.md (The Mechanics)
- **Innovation:** This section explicitly excludes the "Context Hazard" directories (library/), enforcing the PoT.md fencing rule without needing a prompt hack.[1]

Section II: DISPATCH PACKET (The Contract)

- **Why:** To define the "Sandbox."
- **Subsection 2.1: Scope and Safety:**
  - **Objective:** Single-sentence goal.
  - **Allowlist (Target Files):** Explicit list of files. *This replaces the prompt-based*

allowlist.
    - **Negative Constraints:** Explicit list of forbidden actions (e.g., "Do not edit ops/"). *This replaces the prompt-based fencing*.
- **Subsection 2.2: Execution Plan:** Imperative, numbered steps.
- **Subsection 2.3: Results (Formally Robust):** Defines the exact schema for the output artifact.

Section III: THREAD TRANSITION (The Archive)

- **Why:** To manage entropy and "Semantic Debris."
- **Mechanism:** Requires the THREAD END token. The ops/bin/open script will be updated to look for this token. If present, it archives the old task and creates a fresh one. If missing, it refuses to open a new session, enforcing "Equilibrium Maintenance".[1]

7. Implementation Roadmap: Proposal for Change

This section provides the direct response to the user's request for a proposal. It outlines the specific changes required to the TASK.md file and the supporting tooling to enforce the new constitution.

7.1. Proposal: Refactoring TASK.md

The following template replaces the current "weak" file.

STELA TASK DASHBOARD (ACTIVE SURFACE)

Status: ACTIVE

Owner: Integrator

Last Updated: {{TIMESTAMP}}

0. FRESHNESS GATE (Provenance)

- **Active Branch:** {{BRANCH_NAME}}
- **Base HEAD:** {{COMMIT_HASH}}
- **Context Manifest:** ops/lib/manifests/CONTEXT.md
- **Validation:** Run tools/lint/context.sh before proceeding.

I. REQUIRED CONTEXT LOAD (Pointer-First)

Do not hallucinate rules. Load the following pointers:

1. **The Constitution:** PoT.md (Jurisdiction, Filing Doctrine, Axioms).
2. **The Ledger:** SoP.md (History, State of Play).
3. **The Mechanics:** docs/MANUAL.md (Commands, Procedures).
4. **The Terrain:** docs/MAP.md (Continuity, Index).

II. DISPATCH PACKET: {{DP_ID}}

2.1 Scope and Safety (The Contract)

- **Objective:** {{OBJECTIVE_SENTENCE}}
- **Allowlist (Target Files):**
  - {{FILE_PATH}}
- **Negative Constraints (Forbidden):**
  - Do not edit files outside the Allowlist.
  - Do not use contractions (Linguistic Precision).
  - Do not include docs/library/ in global context (Context Hazard).

2.2 Execution Plan (Imperative)

1. **State:** Review active OPEN artifact.
2. **Execution:** Perform logic task {{LOGIC_TASK}}.
3. **Verification:** Run tools/verify.sh.

2.3 Results (Formally Robust)

Populate storage/handoff/{{DP_ID}}-RESULTS.md with:

- **Verification Outputs:** Raw stdout/stderr of tools/lint/truth.sh.
- **Git Status:** Raw output of git status --porcelain.
- **Logic Integrity:** Binary "PASS/FAIL" statement against PoT.md.
- **Drift Report:** Quantitative deviation metric (e.g., "0% deviation").

III. THREAD TRANSITION

- **Status:** PENDING | COMPLETE
- **Next Thread:** {{NEXT_DP_ID}}
- **Closeout:** Append THREAD END marker upon completion.

7.2. Proposal: Deprecating Prompt Hacks

With the new TASK.md structure, the prompts in docs/ops/prompts/ must be stripped of redundant logic.

- **Remove:** The "Logic Fencing" text ("You have no knowledge...") from E-PROMPT-02.
- **Remove:** The "Allowlist Boilerplate" (listing specific directories) from E-PROMPT-03.
- **Replace:** Update all prompts to contain a single "Super-Pointer": *"Execute the Dispatch*

Packet defined in TASK.md. Adhere strictly to the 'Negative Constraints' and 'Allowlist' defined therein."

7.3. Proposal: Hardening the Enforcement Tooling

To ensuring the new TASK.md "enforces itself" (User Query 5), the tools/lint/dp.sh binary must be updated to validate this specific schema.

- **Validation Logic:** The script must grep for the specific headers (## 0. FRESHNESS GATE, ## I. REQUIRED CONTEXT LOAD, ## II. DISPATCH PACKET, ## III. THREAD TRANSITION). Missing headers = Lint Failure.
- **Sanitization Check:** The script must fail if it detects "dirty" placeholders like TBD or [...]. It must only accept ENTER_VALUE or valid content.
- **Head Match:** The script must parse the Base HEAD value in Section 0 and compare it to git rev-parse HEAD. Mismatch = Lint Failure.

7.4. Proposal: The Formally Robust Results Workflow

The "Formally Robust Results" file is not a static template but a generated artifact.

- **Action:** Formalize the generation of storage/handoff/{{DP_ID}}-RESULTS.md.
- **Requirement:** The ops/bin/dump tool should be updated to optionally scaffold this results file at the start of a session, pre-populating the headers (Verification, Git Status) to ensure the agent fills them in correctly.

8. Conclusion and Future Outlook

The current instability of the Stela governance model is a direct result of relying on ephemeral "Prompt Engineering" to enforce what should be immutable "Physical Laws." The "Weak Ass" TASK.md fails to act as a contract, forcing the Operator to rely on prompt hacks that degrade over time and succumb to context saturation.

By implementing the **Hybrid-Hardened TASK.md** proposed in this report, the repository shifts to a "Pointer-First" architecture. In this new state, TASK.md serves as the singular, self-enforcing nexus of governance. It locks the state (Freshness Gate), defines the boundary (Scope/Negative Constraints), and mandates the proof (Formally Robust Results). This eliminates the need for duplicated canon text, resolves the Dirty State Paradox, and establishes a deterministic, autonomous workflow aligned with the Policy of Truth.

Proposal for Change: TASK.md

Refactoring and Logic Hardening

Objective: Harden the TASK.md work surface to enforce PoT.md invariants autonomously, eliminating the reliance on prompt-based fencing hacks, and standardizing the "Formally Robust Results" protocol.

1. Refactor TASK.md to Hybrid-Hardened Schema

Current Status: Weak structure, prone to semantic debris, relies on prompt fencing.

Proposed Change: Replace current TASK.md content with the **Pointer-First Contract Template**.

- **Enforcement:**
  - Adopt **Strict Decimal Schema** (0.0, 1.0, 2.0) to force hierarchy and sequence.
  - Implement **Sanitized Tokens** (ENTER_VALUE) to trigger mechanical stop conditions for missing input.
  - Add **Section 0: Freshness Gate** to enforce Base HEAD validation against the live repo.
  - Add **Section I: Required Context Load** to replace prompt-based context explanations with direct pointers to PoT.md, SoP.md, and docs/MAP.md.
  - Add **Section 2.1: Negative Constraints** to explicitly list forbidden actions (e.g., "Do not edit ops/"), replacing the "Logic Fencing" hack in E-PROMPT-02.

2. Standardize Formally Robust Results

Current Status: Results are generated ad-hoc; lack rigorous definition and quantitative integrity.

Proposed Change: Codify the **RESULTS.md** requirement within TASK.md Section 2.3.

- **Requirement:** Agent must generate storage/handoff/{{DP_ID}}-RESULTS.md.
- **Mandatory Fields:**
  - ## Verification Outputs (Raw): Must contain stdout from tools/lint/*.sh.
  - ## Git Status: Must contain git status --porcelain.
  - ## Logic Integrity: Binary "PASS/FAIL" statement against PoT.md axioms.
  - **Prohibition:** No narrative summaries, "vibes," or conversational filler. Quantitative data only.

3. Elimination of Prompt Hacks

Current Status: E-PROMPT-XX files contain redundant "Do not..." instructions and manual fencing logic.

Proposed Change:

- **Deprecate:** Remove fencing text ("You have no knowledge...") from

docs/ops/prompts/E-PROMPT-02.md.

- **Redirect:** Update prompt to read: *"Execute the Dispatch Packet defined in TASK.md. Adhere strictly to the 'Negative Constraints' and 'Allowlist' defined therein."*
- **Benefit:** Reduces token usage, eliminates "Canon Drift" (rules defined in two places), and centralizes governance in the Contract (TASK.md).

4. Self-Enforcing Policing

Current Status: task_policing.yml checks for updates but not content validity.

Proposed Change:

- **Update:** tools/lint/dp.sh to validate the presence of the new TASK.md sections (Freshness Gate, Negative Constraints, Thread Transition).
- **Workflow:** Ensure task-policing runs tools/lint/dp.sh TASK.md on every PR, rejecting any TASK.md that does not conform to the new schema.

5. Resolution of Dirty State Paradox

Current Status: TASK.md is exempt from dirty state checks but accumulates debris.

Proposed Change:

- **Protocol:** Enforce "Clean After Use" via the **Thread Transition** section.
- **Mechanism:** The presence of THREAD END in TASK.md signals ops/bin/open to archive the current file and generate a fresh template for the next session, programmatically clearing "Semantic Debris."

Final State:

A repository where PoT.md is the only law, TASK.md is the only contract, and prompts are merely triggers. The system enforces itself.

Works cited

1. dump-full-main-4b581538.txt